lamae

BUT2 INFORMATIQUE · RESSOURCE R3.05

Programmation système

CM 05 — Sémaphores, moniteurs et philosophes

```
bash — R3.05
$ gcc -pthread prodcons.c -o prodcons
$ ./prodcons
produit 1   produit 2   consommé 1
produit 3   consommé 2   consommé 3
```

Christophe Cruz · christophe.cruz@ube.fr · 2026-2027

IUT Dijon-Auxerre · Université Bourgogne Europe

RAPPEL

Où en est-on depuis le CM 03b ?

La communication entre processus pose trois problèmes :

01

Transférer des informations

Tubes, files de messages, mémoire partagée (CM 03b, CM 04).

02

Éviter les conflits

Deux processus ne doivent pas être en même temps en section critique. Ex. : deux processus veulent le dernier Mo de mémoire.

03

Respecter un ordre

Si A produit des données que B imprime, B doit attendre A.

Déjà vu au CM 03b : conditions de concurrence, sections critiques, les 4 conditions, et des solutions en attente active : variable verrou (qui échoue), alternance stricte, Peterson, TSL.

Aujourd'hui : des solutions qui endorment les processus au lieu de les faire boucler, puis deux problèmes classiques.

01

sleep et wakeup

Arrêter de boucler pour rien

Dormir plutôt que boucler

- Peterson et TSL sont corrects, mais font de l'attente active : le processus boucle et consomme du CPU
- sleep() : le processus s'endort et rend la main à l'ordonnanceur
- wakeup(p) : réveille le processus p s'il dort
- Idée de base des tubes : un lecteur sur un tube vide dort, l'écrivain le réveille

Attente active



le processus teste en boucle : CPU occupé à 100 %

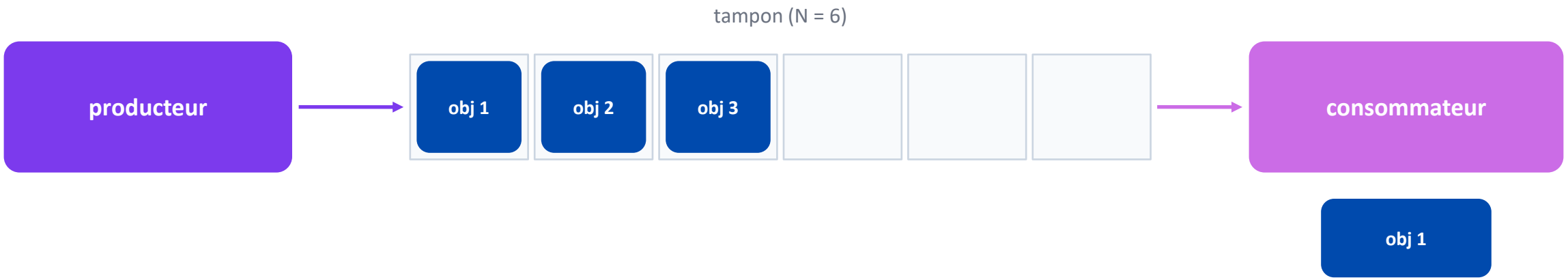
Avec sleep / wakeup

Zzz… endormi, CPU libre pour les autres

wakeup !

Le problème du producteur-consommateur

Un producteur dépose des objets dans un tampon de N cases ; un consommateur les retire. Exemple : un programme qui produit des pages, un démon qui les imprime.



Tampon plein

Le producteur doit s'endormir, jusqu'à ce que le consommateur libère une case.

Tampon vide

Le consommateur doit s'endormir, jusqu'à ce que le producteur dépose un objet.

Une première solution avec sleep / wakeup

producteur

```
#define N 100
int compteur = 0;          // objets dans le tampon

void producteur(void) {
  while (1) {
    objet o = produire();
    if (compteur == N)           // plein
      sleep();
    deposer(o);
    compteur++;
    if (compteur == 1)           // était vide
      wakeup(consommateur);
  }
}
```

consommateur

```
void consommateur(void) {
  while (1) {
    if (compteur == 0)           // vide
      sleep();
    objet o = retirer();
    compteur--;
    if (compteur == N - 1)       // était plein
      wakeup(producteur);
    consommer(o);
  }
}
```

Le wakeup perdu

1. **consommateur :** lit compteur == 0… puis est interrompu
2. **producteur :** dépose un objet, compteur = 1, wakeup(consommateur) : perdu, il ne dort pas !
3. **consommateur :** reprend et exécute sleep() : il s'endort
4. **producteur :** remplit le tampon jusqu'à N, puis sleep()



Les deux processus dorment pour toujours. Cause : l'accès concurrent à compteur, comme pour le spouleur. Tester puis dormir n'est pas atomique.

02

Les sémaphores

Dijkstra, 1965 : compter les réveils

Le sémaphore de Dijkstra

- Un nouveau type de variable : un compteur (de réveils en attente) et une file de processus endormis
- Deux opérations, atomiques : tester, modifier et éventuellement endormir se font en une seule action indivisible
- Le wakeup ne peut plus être perdu : il est mémorisé dans le compteur



down (P)

Si s > 0 : s = s − 1. Sinon : s'endormir dans la file.

up (V)

Si des processus dorment : en réveiller un. Sinon : s = s + 1.

Correction : l'ancien cours écrivait down comme « while (s == 0); s = s - 1; ». C'est de l'attente active, et ce n'est pas atomique : exactement ce que le sémaphore évite.

Un sémaphore en action



1. P1 : down(mutex) → 1 devient 0, P1 entre
2. P2 : down(mutex) → 0 : P2 s'endort
3. P1 : up(mutex) → P2 dort : on le réveille, il entre
4. P2 : up(mutex) → personne n'attend : mutex = 1

L'exclusion mutuelle avec un sémaphore

compte.c

```
semaphore mutex = 1;      // 1 = libre

void retirer(int montant) {
    down(&mutex);         // entrer
    solde = solde - montant;
    up(&mutex);           // sortir
}
```

Le compte bancaire est réparé

Le second retrait attend que le premier ait écrit le solde : 1000 → 900 → 800. On parle de sémaphore binaire, ou mutex.

Sémaphore binaire

Valeur 0 ou 1 : protège une section critique (exclusion mutuelle).

Sémaphore compteur

Valeur de 0 à N : compte des ressources disponibles (cases libres d'un tampon, places d'un parking, connexions…).

Producteur-consommateur avec 3 sémaphores

producteur

```
#define N 100
semaphore mutex = 1;   // accès au tampon
semaphore vides = N;   // cases libres
semaphore pleines = 0; // cases occupées

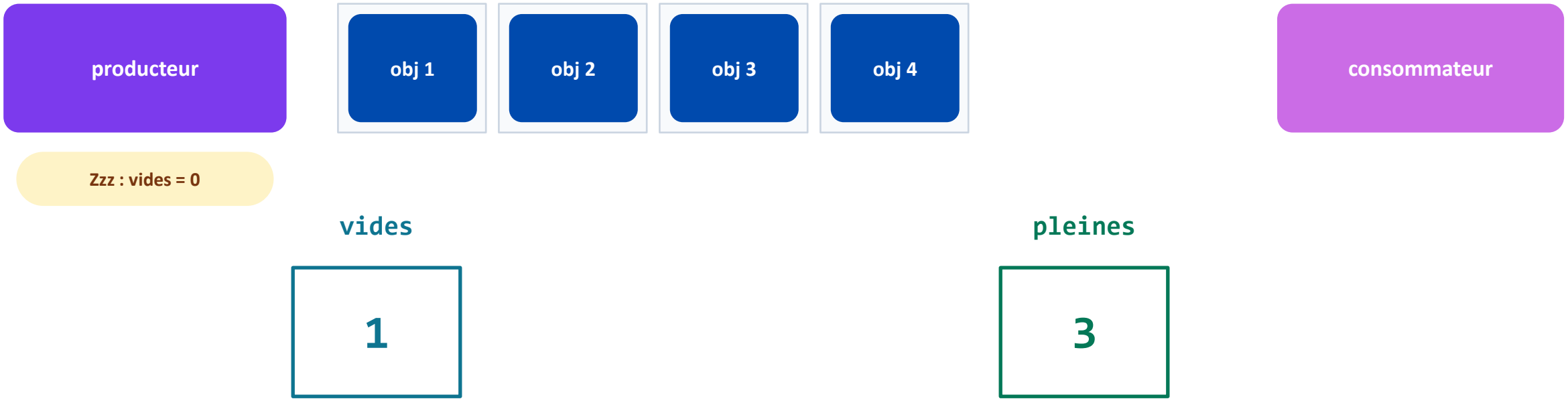
void producteur(void) {
  while (1) {
    objet o = produire();
    down(&vides);
    down(&mutex);
    deposer(o);
    up(&mutex);
    up(&pleines);
  }
}
```

consommateur

Noms corrigés : l'ancienne version appelait « plein » le sémaphore des cases vides.

```
void consommateur(void) {
  while (1) {
    down(&pleines);
    down(&mutex);
    objet o = retirer();
    up(&mutex);
    up(&vides);
    consommer(o);
  }
}
```

Les sémaphores comptent les cases



Invariant : vides + pleines = N (hors des sections critiques). Personne ne teste un compteur à la main : tout est fait par down et up, de façon atomique.

Le piège : inverser les down

producteur (faux)

```
void producteur(void) {
  while (1) {
    objet o = produire();
    down(&mutex);     // inversé !
    down(&vides);
    deposer(o);
    up(&mutex);
    up(&pleines);
  }
}
```

Tampon plein. Le producteur prend mutex…
…puis s'endort sur vides (0), en gardant mutex

Le consommateur veut retirer un objet…
…et s'endort sur mutex, tenu par le producteur

Interblocage : chacun attend l'autre.

Règle : on attend d'abord la ressource (vides ou pleines), puis on prend le mutex le moins longtemps possible. Avec les sémaphores, une simple inversion de deux lignes bloque tout le système.

Les sémaphores POSIX en C

Fonction	Rôle
`sem_init(&s, partage, val)`	initialise un sémaphore anonyme (partage = 0 : threads ; 1 : processus, en mémoire partagée)
`sem_open("/nom", O_CREAT, 0600, val)`	crée ou ouvre un sémaphore nommé, utilisable par des processus sans lien
`sem_wait(&s)`	down (P)
`sem_post(&s)`	up (V)
`sem_destroy / sem_close / sem_unlink`	libérer

compiler avec -pthread

```c
#include <semaphore.h>
sem_t vides, pleines, mutex;
sem_init(&vides, 0, N);
sem_init(&pleines, 0, 0);
sem_init(&mutex, 0, 1);
/* producteur : */ sem_wait(&vides); sem_wait(&mutex); /* déposer */ sem_post(&mutex); sem_post(&pleines);
```

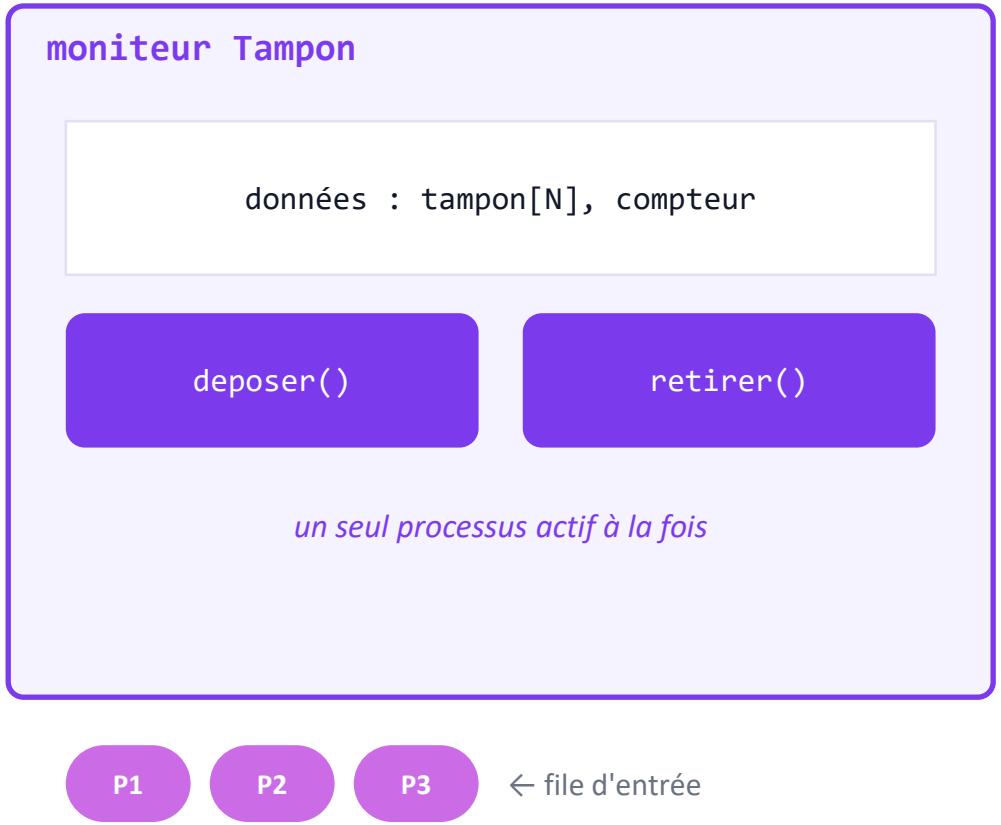
03

Les moniteurs

Laisser le langage gérer l'exclusion mutuelle

Le moniteur (Hoare, Brinch Hansen, 1974)

- Les sémaphores sont délicats : un down oublié, deux down inversés… et tout se bloque
- Un moniteur regroupe des données et les procédures qui les manipulent
- Les processus n'accèdent aux données que par ces procédures
- À un instant donné, un seul processus est actif dans le moniteur : l'exclusion mutuelle est garantie par le langage
- Exemple : synchronized en Java



Attendre dans un moniteur : les variables de condition

- Le moniteur garantit l'exclusion, mais il faut aussi pouvoir attendre (tampon plein ou vide)
- wait(c) : le processus s'endort sur la condition c et libère le moniteur
- signal(c) : réveille un processus endormi sur c

Plus de wakeup perdu : tester compteur et faire wait se passe dans le moniteur, donc sans interruption par un autre processus du moniteur.

pseudo-code (Tanenbaum)

```
monitor Tampon {
  condition plein, vide;
  int compteur = 0;

  void deposer(objet o) {
    if (compteur == N) wait(plein);
    /* ranger o */  compteur++;
    if (compteur == 1) signal(vide);
  }
  objet retirer(void) {
    if (compteur == 0) wait(vide);
    /* prendre o */ compteur--;
    if (compteur == N - 1) signal(plein);
    return o;
  }
}
```

Un moniteur en C : mutex + condition

moniteur.c

```c
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  pas_plein = PTHREAD_COND_INITIALIZER;
pthread_cond_t  pas_vide  = PTHREAD_COND_INITIALIZER;

void deposer(objet o) {
    pthread_mutex_lock(&m);              // entrer dans le « moniteur »
    while (compteur == N)                // while, pas if !
        pthread_cond_wait(&pas_plein, &m);
    ranger(o); compteur++;
    pthread_cond_signal(&pas_vide);
    pthread_mutex_unlock(&m);            // sortir
}
```

Java : synchronized + wait() / notify(). C'est le même modèle.

pthread_cond_wait

Libère le mutex et s'endort, de façon atomique ; le reprend au réveil.

Pourquoi while ?

Un réveil peut être intempestif, ou un autre processus peut être passé avant : on reteste toujours la condition.

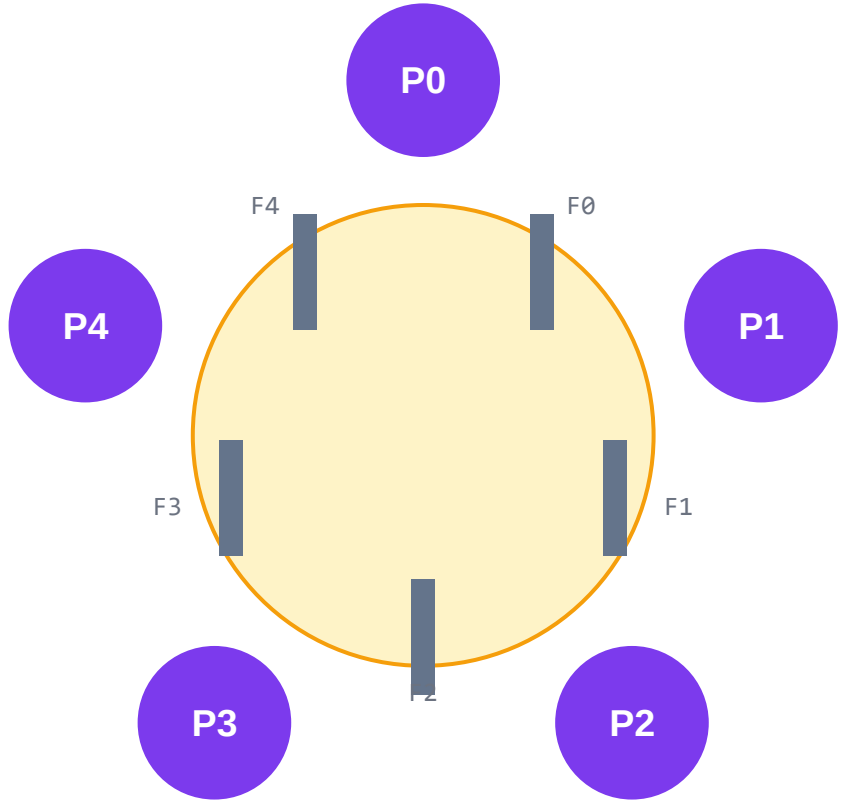
04

Le dîner des philosophes

Dijkstra, 1965 : interblocage et famine

Le problème

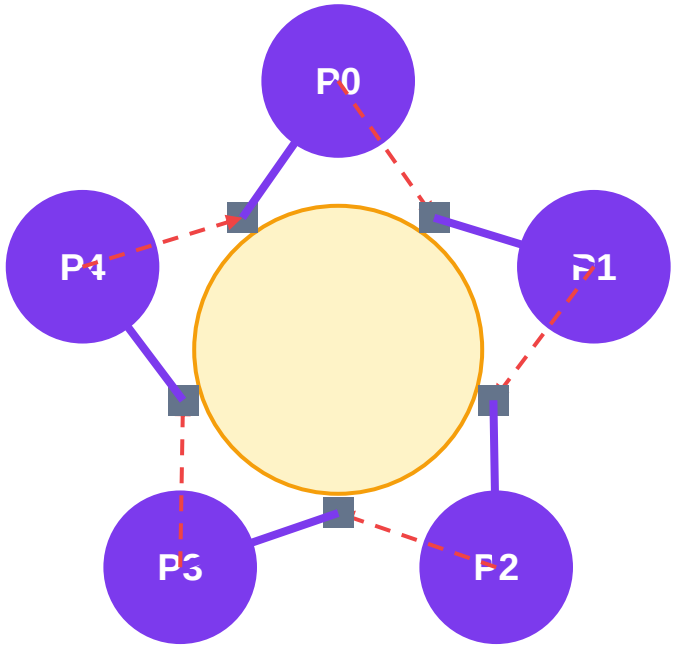
- 5 philosophes autour d'une table : ils pensent ou mangent
- Pour manger, il faut 2 fourchettes : celle de gauche et celle de droite
- Il n'y a que 5 fourchettes, une entre chaque voisin
- Le problème illustre l'allocation de ressources sans interblocage ni famine



La solution naïve mène à l'interblocage

naif.c

```c
void philosophe(int i) {
  while (1) {
    penser();
    prendre(i);              // gauche
    prendre((i + 1) % 5);    // droite
    manger();
    poser(i);
    poser((i + 1) % 5);
  }
}
```



Chacun tient sa fourchette gauche et attend la droite, tenue par son voisin : attente circulaire, personne ne mange. Reposer et réessayer ne suffit pas : ils peuvent recommencer ensemble indéfiniment (famine).

Une solution avec les états

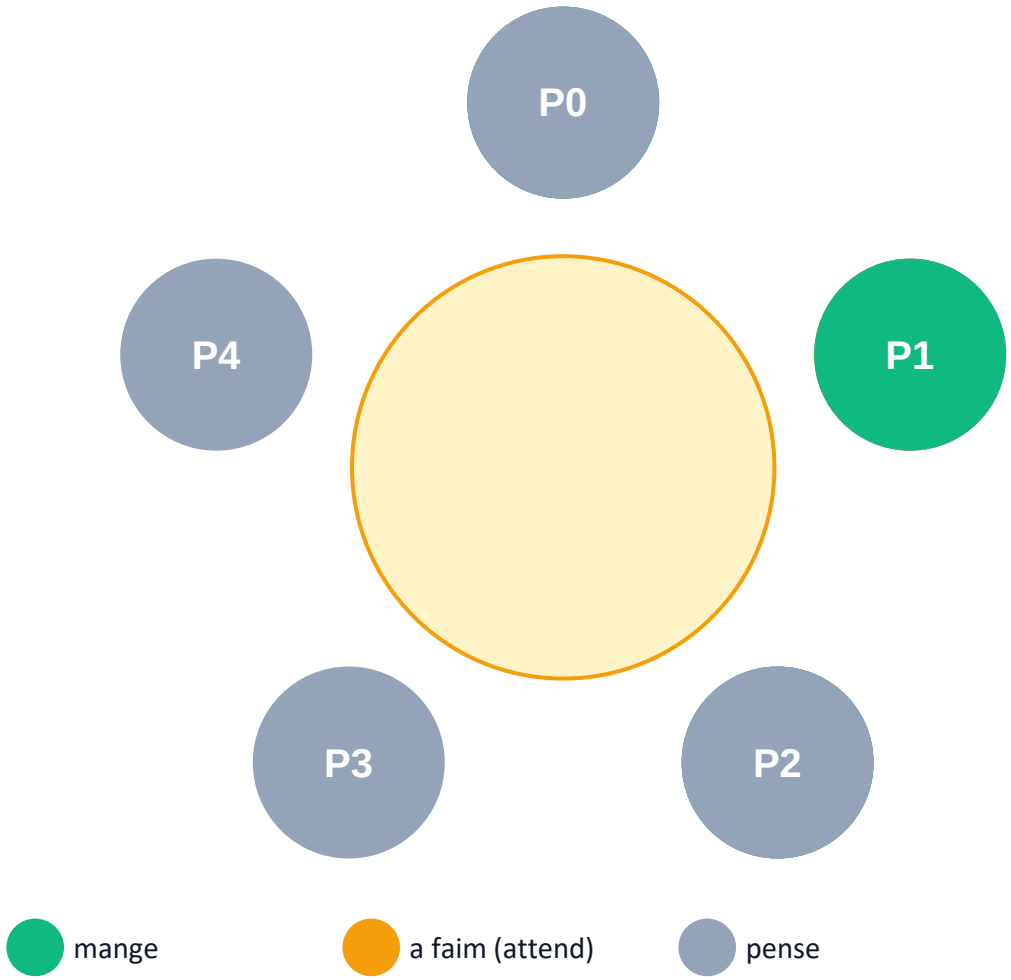
philosophes (moniteur)

```
enum { PENSE, A_FAIM, MANGE } etat[5];
#define G(i) (((i) + 4) % 5)     // voisin de gauche
#define D(i) (((i) + 1) % 5)     // voisin de droite

void tester(int i) {
  if (etat[i] == A_FAIM && etat[G(i)] != MANGE
                        && etat[D(i)] != MANGE) {
    etat[i] = MANGE;
    signal(self[i]);             // réveiller i
  }
}
void prendre_fourchettes(int i) {   // dans le moniteur
  etat[i] = A_FAIM;
  tester(i);
  if (etat[i] != MANGE) wait(self[i]);
}
void poser_fourchettes(int i) {
  etat[i] = PENSE;
  tester(G(i));                  // mes voisins peuvent-ils
  tester(D(i));                  // manger maintenant ?
}
```

- Un philosophe ne prend ses fourchettes que s'il peut prendre les deux : jamais une seule
- Il mange si ses deux voisins ne mangent pas
- En reposant, il teste ses voisins : ceux qui avaient faim peuvent commencer
- Chaque philosophe exécute : penser, prendre_fourchettes, manger, poser_fourchettes

La solution en action



1. P0 a faim, ses voisins pensent : il mange
2. P1 a faim, mais P0 mange : il attend
3. P2 a faim : P1 et P3 ne mangent pas, il mange
4. P0 pose et teste P1 : P2 mange encore, P1 attend
5. P2 pose et teste P1 : P1 mange enfin

Interblocage : les 4 conditions de Coffman (1971)

Un interblocage ne peut se produire que si les quatre conditions sont réunies. Il suffit d'en casser une.

01

Exclusion mutuelle

Une fourchette ne sert qu'à un philosophe à la fois.

02

Détention et attente

Il garde la gauche en attendant la droite. → Tout prendre d'un coup (solution par états).

03

Pas de réquisition

On ne peut pas lui retirer sa fourchette de force.

04

Attente circulaire

P0 attend P1 qui attend P2… qui attend P0. → Numéroter les fourchettes et prendre toujours la plus petite d'abord, ou n'admettre que 4 philosophes à table (sémaphore à 4).

À retenir

01

sleep / wakeup

Dormir plutôt que boucler. Seul, cela perd des réveils : il faut une opération atomique.

02

Sémaphore

Compteur + file. down endort si 0, up réveille ou incrémente. Atomiques.

03

Producteur-consommateur

vides = N, pleines = 0, mutex = 1. Attendre la ressource avant le mutex.

04

Moniteur

Exclusion garantie par le langage. Variables de condition : wait libère le moniteur.

05

En C

sem_t (sem_wait, sem_post), pthread_mutex + pthread_cond, avec while.

06

Philosophes

Interblocage = 4 conditions de Coffman. En casser une suffit.

Ressources pédagogiques

Le manuel

```
$ man 7 sem_overview
$ man 3 sem_wait
$ man 3 pthread_cond_wait
$ man 3 pthread_mutex_lock
```

À la séance 8 (threads et mutex), vous programmerez le producteur-consommateur et les philosophes avec pthread.

Lectures

- A. Tanenbaum, Systèmes d'exploitation : « Communication interprocessus » et « Interblocages »
- A. B. Downey, The Little Book of Semaphores (gratuit en ligne) : des dizaines de problèmes de synchronisation
- E. W. Dijkstra, Cooperating Sequential Processes, 1965

Questions ?

PROCHAINE SÉANCE

CM 06 — L'ordonnanceur

FIFO, SJF, tourniquet, priorités · diagrammes de Gantt · l'ordonnanceur de Linux

Christophe Cruz · christophe.cruz@ube.fr